COMP9024 19T3 Data Structures and Algorithms

Week 9 Problem Set
String Algorithms, Approximation

1. (Boyer-Moore algorithm)

   a. Implement a C-function

   ```
   int *lastOccurrence(char *pattern, char *alphabet) { ... }
   ```

   that computes the last-occurrence function for the Boyer-Moore algorithm. The function should return a newly created dynamic array indexed by the numeric codes of the characters in the given alphabet (a non-empty string of ASCII-characters).

   Ensure that your function runs in *O(m+s)* time, where *m* is the size of the pattern and *s* the size of the alphabet.

   Hint: You can obtain the numeric code of a `char c` through type conversion: `(int)c`.

   b. Use your answer to Exercise a. to write a C-program that:

   - prompts the user to input
     - an alphabet (a string),
     - a text (a string),
     - a pattern (a string);
   - computes and outputs the last-occurrence function for the pattern and alphabet;
   - uses the Boyer-Moore algorithm to match the pattern against the text.

   An example of the program executing could be

   ```
   prompt$ ./boyer-moore
   Enter alphabet: abcd
   Enter text: abacaabadcabacabaabb
   Enter pattern: abacab

   L[a] = 4
   L[b] = 5
   L[c] = 3
   L[d] = -1

   Match found at position 10.
   ```

   If no match is found the output should be: `No match.`

   Hints:

   - You may assume that
     - the pattern and the alphabet have no more than 127 characters;
     - the text has no more than 1023 characters.
   - To scan `stdin` for a string with whitespace, such as `"a pattern matching algorithm"`, you can use:

     ```
     #define MAX_TEXT_LENGTH 1024
     #define TEXT_FORMAT_STRING "%[^\n]%*c"

     char T[MAX_TEXT_LENGTH];
     scanf(TEXT_FORMAT_STRING, T);
     ```

     This will read every character as long as it is not a newline `'\n'`, and `"%*c"` ensures that the newline is read but discarded.

We have created a script that can automatically test your program. To run this test you can execute the `dryrun` *program that corresponds to this exercise. It expects to find a program named* `boyer-moore.c` *in the current directory. You can use autotest as follows:*

```
prompt$ 9024 dryrun boyer-moore
```

Answer:

a.

```
#define ASCII_SIZE 128

int *lastOccurrenceFunction(char *pattern, char *alphabet) {
   int *L = malloc(ASCII_SIZE * sizeof(int));
   assert(L != NULL);
```

```
   int i, s = strlen(alphabet);
   for (i = 0; i < s; i++)             // for all chars in the alphabet
      L[(int)alphabet[i]] = -1;        // ... initialise L[] to -1

   int m = strlen(pattern);
   for (i = 0; i < m; i++)
      L[(int)pattern[i]] = i;          // set L[]

   return L;
}
```

b.

```
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include <string.h>

#define MAX_TEXT_LENGTH 1024
#define MAX_PATTERN_LENGTH 128
#define MAX_ALPHABET_LENGTH 128
#define TEXT_FORMAT_STRING "%[^\n]%*c"
#define PATTERN_FORMAT_STRING "%[^\n]%*c"
#define ALPHABET_FORMAT_STRING "%[^\n]%*c"

#define MIN(x,y) ((x < y) ? x : y)    // ternary operator (cond ? t1 : t2)
                                      // => evaluates to t1 if (cond)≠0, else to t2

int boyerMoore(char *text, char *pattern, int *L) {
   int n = strlen(text);
   int m = strlen(pattern);

   int i = m-1;
   int j = m-1;
   do {
      if (text[i] == pattern[j]) {
         if (j == 0) {
            return i;
         } else {
            i--;
            j--;
         }
      } else {                            // character jump
         i = i + m - MIN(j, 1+L[(int)text[i]]);
         j = m - 1;
      }
   } while (i < n);
   return -1;                             // no match
}

int main(void) {
   char T[MAX_TEXT_LENGTH];
   char P[MAX_PATTERN_LENGTH];
   char S[MAX_ALPHABET_LENGTH];

   printf("Enter alphabet: ");
   scanf(ALPHABET_FORMAT_STRING, S);
   printf("Enter text: ");
   scanf(TEXT_FORMAT_STRING, T);
   printf("Enter pattern: ");
   scanf(PATTERN_FORMAT_STRING, P);
   putchar('\n');

   int *L = lastOccurrenceFunction(P, S);
   int i, s = strlen(S);
   for (i = 0; i < s; i++)
      printf("L[%c] = %d\n", S[i], L[(int)S[i]]);

   int match = boyerMoore(T, P, L);
   free(L);
   if (match > -1)
      printf("\nMatch found at position %d.\n", match);
   else
```

```
        printf("\nNo match.\n");

    return 0;
}
```

2. (Knuth-Morris-Pratt algorithm)

   Develop, in pseudocode, a modified KMP algorithm that finds *every* occurrence of a pattern *P* in a text *T*. The algorithm should return a queue with the starting index of every substring of *T* equal to *P*.

   Note that your algorithm should still run in *O(n+m)* time, and it should find every match, including those that "overlap".

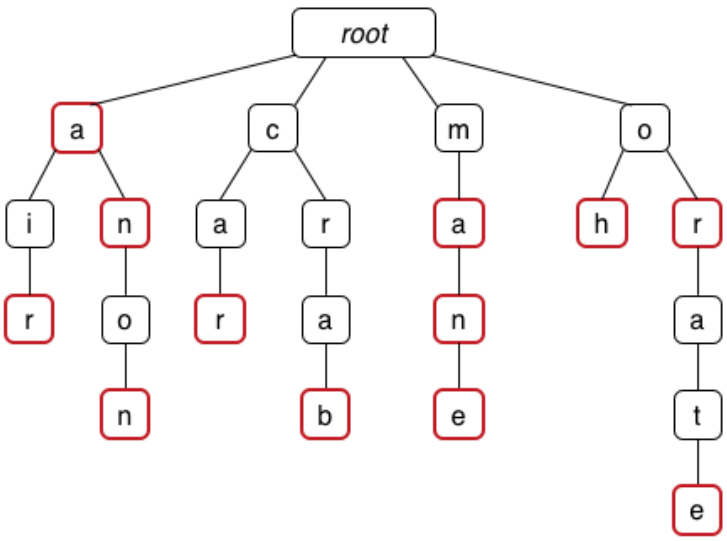
   Answer:

```
KMPMatchAll(T,P):
   Input  text T of length n, pattern P of length m
   Output queue with all starting indices of substrings of T equal to P

   F=failureFunction(P)
   i=0, j=0
   Q=empty queue
   while i<n do
      if T[i]=P[j] then
         if j=m-1 then
            enqueue i-j into Q     // match found
            i=i+1, j=F[m-1]        // continue to search for next match
         else
            i=i+1, j=j+1
         end if
      else
         if j>0 then
            j=F[j-1]
         else
            i=i+1
         end if
      end if
   end while
   return Q                        // if Q is empty, no match found
```

3. (Tries)

   a. Consider the following trie, where finishing nodes are shown in red:



   What words are encoded in this trie?

   b. If the following keys were inserted into an initially empty trie:

```
jaws boots axe boo so jaw sore boot boon
```

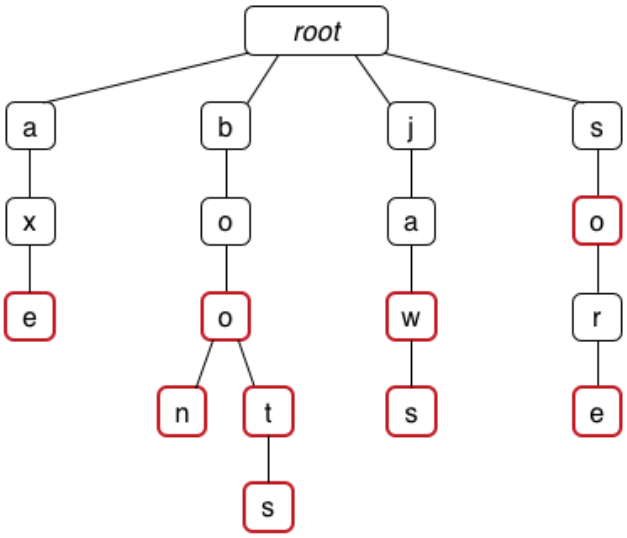
   what would the final trie look like? Does the order of insertion matter?

   c. Answer question b. for a compressed trie.

Answer:

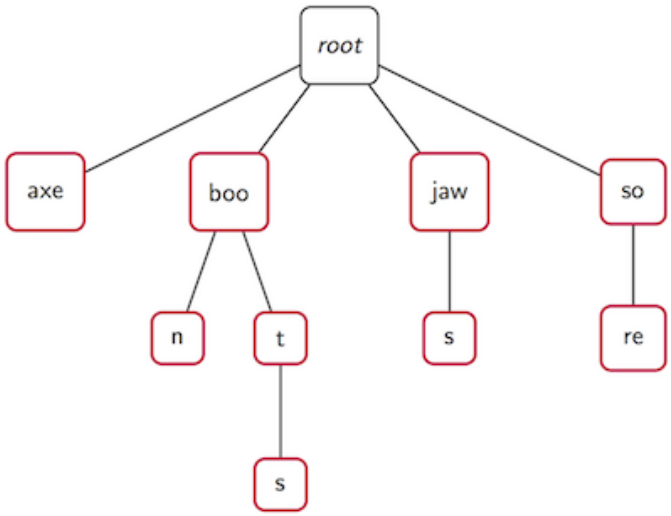
a. In alphabetical order: a, air, an, anon, car, crab, ma, man, mane, oh, or, orate.

b. The trie after all keys are inserted:



The order of insertion does not matter. The same trie will always result from insertion of the same set of words.

c. The compressed trie after all keys are inserted:



Again, the order of insertion does not matter.

4. (Text compression)

Compute the frequency array and draw a Huffman tree for the following string:

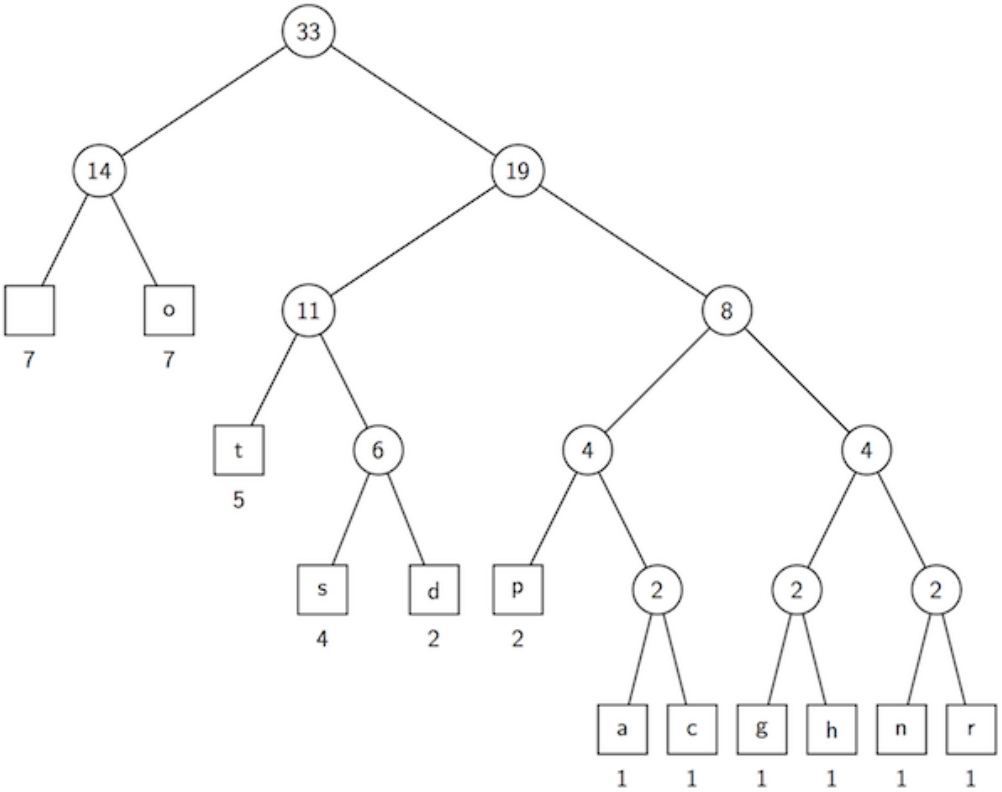
```
dogs do not spot hot pots or cats
```

Answer:

The frequency array:

Character		a	c	d	g	h	n	o	p	r	s	t
Frequency	7	1	1	2	1	1	1	7	2	1	4	5

A Huffman tree:



Other solutions are possible.

5. (Numerical approximation)

Write a C program to implement the root finding approximation algorithm from the lecture as the function:

```
double bisection(double (*f)(double), double x1, double x2)
```

Use your program to find roots for

a. $f(x) = x^3 - 7x - 6$, in the interval $[0.0, 10.0]$

b. $f(x) = \sin x$, in the interval $[2.0, 4.0]$

c. $f(x) = \sin 5x + \cos 10x + \frac{x^2}{10}$, in the intervals $[0.0, 1.0]$ and $[1.0, 2.0]$

with precision $\varepsilon = 10^{-10}$.

Answer:

```
#include <stdlib.h>
#include <stdio.h>
#include <math.h>

#define EPSILON 1.0e-10

double bisection(double (*f)(double), double x1, double x2) {
   double mid;
   do {
      mid = (x1+x2) / 2;
      if (f(x1) * f(mid) < 0)  // root to the left of mid
         x2 = mid;
      else                     // root to the right of mid
         x1 = mid;
   } while (f(mid) != 0 && x2-x1 >= EPSILON);
   return mid;
}

double f1(double x) {
   return pow(x,3) - 7*x - 6;
}
```

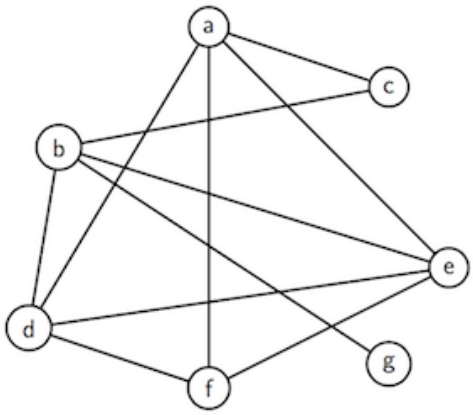
```
double f3(double x) {
   return sin(5*x) + cos(10*x) + x*x/10;
}

int main(void) {
   printf("%.10f\n", bisection(f1, 0, 10));
   printf("%.10f\n", bisection(sin, 2, 4));
   printf("%.10f\n", bisection(f3, 0, 1));
   printf("%.10f\n", bisection(f3, 1, 2));
   return 0;
}
```

a. 3.0
b. π (= 3.1415926536)
c. 0.7371977788 and 1.1420071707

6. (Vertex cover)

a. Apply the approximation algorithm for vertex cover to the following graph:



Find two different executions, one that results in an optimal cover and one that does not.

b. What size is an optimal vertex cover for complete graphs K_n? Does the approximation algorithm always find an optimal cover for complete graphs?

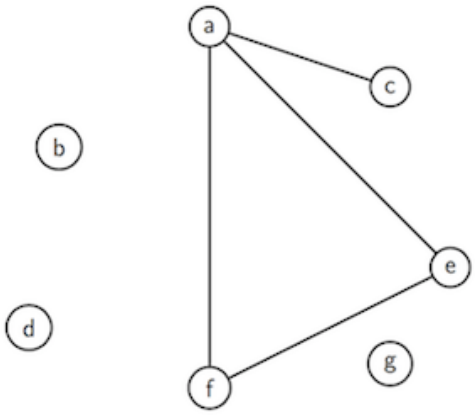
c. A *complete bipartite graph* $K_{m,n}$ is an undirected graph that has:

- two disjoint vertex sets V_m and V_n of size $m \geq 1$ and $n \geq 1$, respectively;
- every possible edge connecting a vertex in V_m to a vertex in V_n;
- no edge that has both endpoints in V_m or both endpoints in V_n.

Answer question b. for complete bipartite graphs.

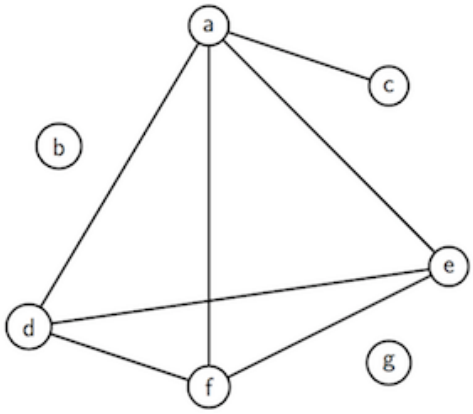
Answer:

a. In the first iteration of the approximation algorithm for vertex cover, the edge b-d may be chosen. We remove all the edges that are incident on b or d:

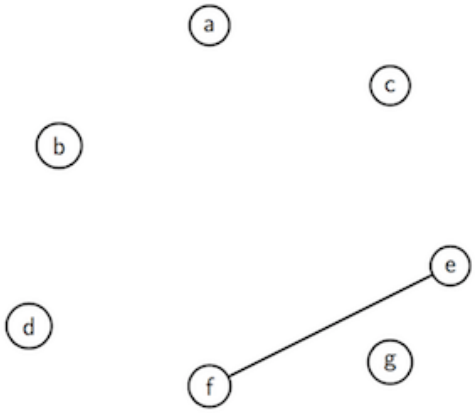


In the second iteration, the edge a-e may be chosen. This will remove all the remaining edges, and the result is an optimal vertex cover: {a,b,d,e}.
Note: There are different ways to cover this graph with 4 vertices. But there is no vertex cover of smaller size.

In another run of the algorithm, the edge b-g may be chosen in the first iteration. We remove all the edges that are incident on b or g:



In the second iteration, the edge a-d may be chosen. We remove all the edges incident on a or d:



Since all edges have not been used, we must go through another iteration and choose the remaining edge e-f. This results in the non-minimal vertex cover {a,b,d,e,f,g}.

b. Complete graphs K_n have an optimal vertex cover of size *n-1*: Choose any *n-1* vertices, then obviously every edge in the graph is covered by at least one of the vertices. And if you take fewer than *n-1* vertices, then the edge between any two nodes that you did not choose would not be covered.

   The approximation algorithm always returns an even number of vertices. Hence, it only finds an optimal cover for complete graphs K_n with an *odd* number of nodes, that is, K_3, K_5, …. (Technically this also includes K_1, i.e. a graph with a single node and no edge, whose optimal vertex cover is the empty set.)

c. Bipartite graphs $K_{m,n}$ have an optimal vertex cover of size $min\{m,n\}$: Without loss of generality, assume $m \leq n$. Then V_m is a vertex cover since every edge is adjacent to some vertex in V_m. Moreover, there can be no smaller vertex cover. For if you try a proper subset of V_m, and even if you add to it a proper subset of V_n, then the edge between any two missing nodes from these subsets would not be covered.

   The approximation algorithm will always find a cover *twice* the minimum size. For it always picks both endpoints of an edge, one of which comes from the larger of the two disjoint sets of vertices. Hence the size of a computed vertex cover is $2 \cdot min\{m,n\}$.

7. (Feedback)

   We (Michael and Michael, Michael and Shanush) want to hear from you how you liked COMP9024 and if you have any suggestions on how the course should be run in the future.

   Log in to MyExperience to provide feedback. Please do so even if you just want to say, "I liked the way it was taught".

8. **Challenge Exercise**

   Given a string *s* with repeated characters, design an efficient algorithm for rearranging the characters in *s* so that no two adjacent characters are identical, or determine that no such permutation exists. Analyse the time complexity of your algorithm.

   Answer:

   The problem can be solved with a greedy approach: In each step, we select a character with the highest frequency that is different from the character selected before. Every time a character is selected, its frequency is reduced by one.

```
rearrangeString(S):
   Input  string S
   Output permutation of S such that no two adjacent chars are the same
          false if no such permutation exists

   compute frequency of each char in S
   P=priority queue of distinct chars in S with frequency as key
   S_new=empty string
   c=leave(P), append c to S_new, c.key=c.key-1
   while P is not empty do
      d=leave(P), append d to S_new, d.key=d.key-1
      if c.key>0 then
         join(P,c)         // insert c back into the priority queue
      end if
      c=d
   end while
   if c.key>0 then
      return false
   else
      return S_new
   end if
```

Time complexity analysis:

Let *n* be the size of the input string *s*.

1. Computing the frequencies of all characters in *s* takes *O(n)* time.
2. Creating a priority queue for all distinct characters using a self-balancing BST takes *O(n·log n)* time.
3. Using a self-balancing BST, both `leave()` and `join()` take *O(log n)* time. Hence, the while-loop takes *O(n·log n)* time.

Therefore, the complexity of the algorithm is *O(n·log n)*.

Assessment

After you've solved the exercises, go to COMP9024 19T3 Quiz Week 9 to answer 5 quiz questions on this week's problem set (Exercises 1-7 only) and lecture.

The quiz is worth 2 marks.

The deadline for submitting your quiz answers is **Monday, 18 November 11:00:00am**.

As always please be mindful of the **quiz rules**:

Do …

- use your own best judgement to understand & solve a question
- discuss quizzes on the forum only **after** the deadline on Monday

Do not …

- post specific questions about the quiz **before** the Monday deadline
- agonise too much about a question that you find too difficult